

Lab Assignment 4

Christos Strydis
Rajendra Bishnoi
Amirreza Movahedin

2026

Contents

1	Introduction	2
2	Some Practical Theory and Techniques Behind AI	2
3	Image Classification	3
3.1	Multi-Layer Perceptron (MLP)	4
3.2	Convolutional Neural Network (CNN)	5

Submission guidelines:

- Hand in the lab report in pdf format with the following name:
Lab4.Firstname.Lastname.Studentnumber (for example: Lab4.Pieter.deBoer.123456)
- Hand in the Python scripts (the **.py** files) as separate, unzipped Python scripts. Do not forget to add meaningful comments to your code. Handing in poorly annotated code will be penalized with one point.
- You must submit your code for exercises **4.4** and **4.6** alongside your report for this lab. Please name your script files with the following convention: **L4.Qx** where x is the exercise number.

1 Introduction

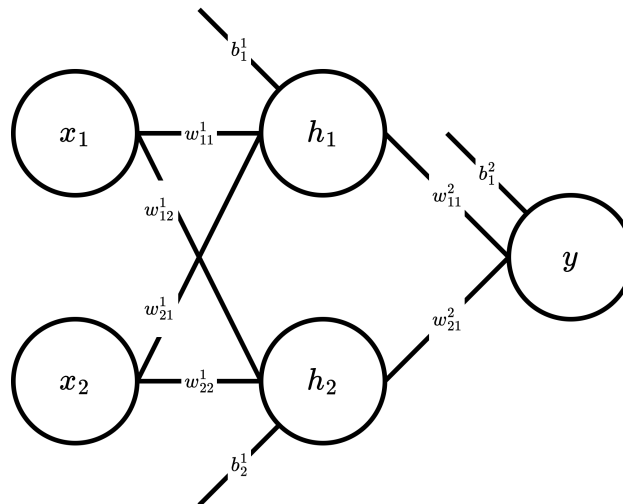
In the next two lab assignments, we shift our focus from the HPC part of the course onto the Artificial Intelligence (AI) part. Although we should keep in mind that these two parts are not unrelated at all. The goal of this lab is to dive into the fundamental concepts of AI training and inference and to train different AI models for different tasks. First, we will review some theoretical concepts, then we will develop models for image classification tasks, and lastly, we will look into AI models for sequential data.

2 Some Practical Theory and Techniques Behind AI

In this section, we are going over some basic theoretical concepts of AI. Some answers might be found in lecture slides, some other might require you to search around and read some blog posts or papers.

Exercise 4.1 (7.5 points)

Consider the below simple feed-forward neural network.



The output neuron (y) can be described based on the input neurons (x_1 and x_2) as follows:

$$h_1 = \text{ReLU}(w_{11}^1 x_1 + w_{21}^1 x_2 + b_1^1)$$

$$h_2 = \text{ReLU}(w_{12}^1 x_1 + w_{22}^1 x_2 + b_2^1)$$

$$y = \text{sigmoid}(w_{11}^2 h_1 + w_{21}^2 h_2 + b_1^2)$$

In order to train this network, we try to optimize its parameters (w and b). A very commonly used technique is *back propagation*. During back propagation, the gradient of a loss function with respect to each parameter is computed by repeatedly applying the chain rule from the output layer back to the input layer, and the parameters are updated in the direction that minimizes the loss.

1. Let's imagine the simple loss function of mean squared error (MSE), $\mathcal{L}(y) = \frac{1}{2}(y - \hat{y})^2$, for this network ($\hat{y}$ is a constant value). Using the chain rule, calculate the gradient of the loss function with respect to w_{11}^2 and w_{21}^2 ($\frac{\partial \mathcal{L}}{\partial w_{11}^2}$ and $\frac{\partial \mathcal{L}}{\partial w_{21}^2}$). (5 points)
2. As you saw, calculating gradients by hand, even for a very simple network, is quite a challenging task. For much deeper or more complicated networks, this task quickly becomes intractable and error-prone. To address this challenge, automatic differentiation frameworks were introduced. Briefly explain these frameworks and how they work, in addition to how they help with this challenge. (2.5 points)

Exercise 4.2 (5 points)

Consider a fully-connected, feed-forward neural network (or an MLP) with I input neurons, L hidden layers with H neurons each, and O output neurons.

1. What is the total number of parameters of this neural network? (2.5 points)
2. Is this number of parameters and computation needed feasible for more challenging tasks which require larger number of neurons and deeper networks? How do convolutional neural networks (CNNs) addressed this challenge? (2.5 points)

Exercise 4.3 (10 points)

As mentioned earlier, during the training of a network, the gradient of the loss function with respect to the parameters are calculated and the parameters are adjusted towards minimizing the loss function. The learning happen using the training dataset, where each data element contributes to the adjustment of the parameters.

$$w_{ij}^{t+1} = w_{ij}^t - \alpha \frac{\partial \mathcal{L}}{\partial w_{ij}}$$

1. In practice, the training dataset is divided into smaller batches, and the average gradient of each batch is applied to adjust the weights; an epoch corresponds to one complete pass through the entire training dataset, during which all batches are processed once. Why do you think dataset batching is performed? Do you think different ways of dividing the dataset into smaller batches affect the training? (2.5 points)
2. In the above formula, α refers to the learning rate. It is an adjustable hyper-parameter of the training process that dictates how significant is the adjustment to the parameters. What would happen if the learning rate is too large or too small, and how can that harm the training process? In practice, the learning rate is scheduled. What does that mean and how it can improve the training process? (2.5 points)
3. We train the network on training dataset. However, the accuracy of the network is measured using the test dataset. The concepts of over- and underfitting refer to the discrepancy of the network's performance on seen and unseen data. What do these phenomenon mean, and what are some the signs of them? (2.5 points)
4. During training, we try to minimize the loss function value. However, lower loss function value does not necessarily mean higher test accuracy of the network. What do you think the loss function value represent? How can it relate to overfitting? (2.5 points)

3 Image Classification

In this section, we are going to perform an image classification task using two different types of networks: MLPs and CNNs. The task is hand-written digit recognition using the MNIST dataset, which consists of 28×28 grayscale images of handwritten digits (0–9). The images are fed into the network, and the network has to classify them into which digit they are.



In order to build these networks, we are using the JAX framework, in addition to a higher level JAX-based library named Flax. While this lab uses JAX as the implementation framework, other widely used frameworks such as PyTorch or TensorFlow also exist. The choice of framework is not central to this lab; rather, the objective is to introduce the fundamental concepts and practical mechanics of AI model creation and training, which are largely framework-independent.

3.1 Multi-Layer Perceptron (MLP)

As you have seen in the lectures, MLPs are fully-connected, multi-layer neural networks. In this section, we are gonna build and train an MLP to perform the MNIST image classification task. Open the `mlp.py` file which consists of a simple MLP training code using JAX+Flax. At first, we load the data using the TensorFlow datasets, and normalize their values between 0 and 1 in addition to flattening the arrays to feed to the network. Then, we define our MLP class which contains the computations that the network performs, as you can see in the `__call__` function. You can see how the Flax library makes defining these models easy. Note that the MLP model is stateless, meaning it does not hold its parameters within it and only contains the computations performed in the network.

After that, we create an instance of the model and initialize the parameters to random values. Then we define the loss function, which is a softmax cross entropy. The next function is `predict`, which is the forward pass and prediction of the network. The `accuracy` function runs the model for a set of data and reports the accuracy of the model. Lastly, we define the `training_step` function, which consists of the processes we do at each step of training. First, the gradients of the loss function with regards to the current parameters are calculated. Then, the parameters are updated based on the gradients and the learning rate and the new parameters are returned. And finally, in the nested loops, we perform the training steps for a certain number of epochs.

Exercise 4.4 (17.5 points)

Run the code and try to understand it.

1. What is the trained and untrained accuracy of the model? Do these results change when you change the seed to the PRNG key generator? (the `key = random.PRNGKey(SEED)` line) (2.5 points)
2. Comment out the normalization performed on the dataset at the beginning of the code and report your observation. Why do you think is the reason for the difference in accuracy? (2.5 points)
3. Run the training for 10 epochs for learning rates of (0.1, 0.0001, 0.0000001) and report your observations. What happens with too large or too small learning rates? (2.5 points)
4. The current code does not perform batching for training. Implement the batching of the training dataset for a variable batch size in the training loop. Report the final accuracy of the model for batches of 32, 64, 128, 256, and 512 for 10 epochs of training. How do they compare to the previous results? (5 points)
5. What happens if you increase the learning rate for the batched training with 64 batch size? How do you explain your observation? (2.5 points)
6. Reduce the size of the training set to 1000 samples and run the training for 20 epochs with batch size of 64. What are the training and test accuracy results of the model? What phenomenon are you observing? (2.5 points)

Exercise 4.5 (12.5 points)

The `mlp.py` file contains a simple yet working model creation and training for the MNIST task. However, there are much more techniques used in neural network training, and implementing all of them manually is not optimal. We showed a more standard model creation and training procedure in the `mlp.adv.py` file that includes extra techniques and regularizations.

1. Look at the MLP class definition. You can see that this MLP has two hidden layers of 512 neurons each. Additionally, there are dropout layers added to the model which only activates during training. What is dropout in neural network training? (2.5 points)
2. Do you think it is beneficial to have even more hidden layers of more neurons each? (2.5 points)

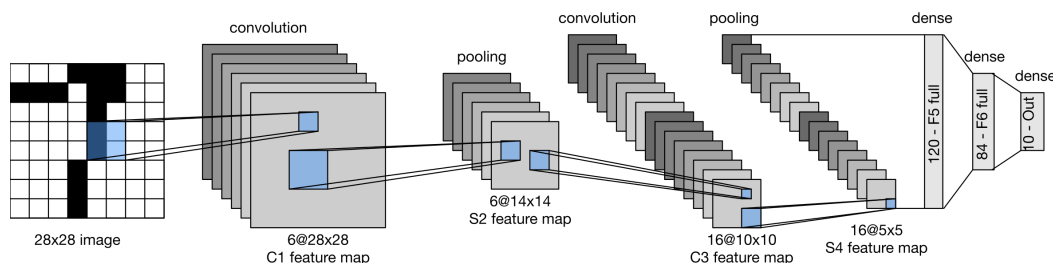
- After the initiation of the model and the parameters, you can see an optimizer has been added in this code. You know about the learning rate as part of the optimizer. What is the `adamw` optimizer and what role does the weight decay parameter plays in it? (2.5 points)
- In this code, you can also see changes in the loss and training step functions which mostly attributes to the addition of dropout and the optimizer. Run this version of the code for 3 different seeds for 10 epochs and report the accuracy results. How do they compare to the previous version of the code? (5 points)

3.2 Convolutional Neural Network (CNN)

CNNs are a class of neural networks designed to process data with an explicit spatial structure, such as images. Unlike MLPs, which treat each input dimension independently, CNNs exploit two key properties of images: locality (nearby pixels are more related than distant ones) and translation invariance (the same feature can appear anywhere in the image). CNNs achieve this through convolutional layers, which apply small, learnable filters across the input using shared weights. This drastically reduces the number of free parameters compared to dense models and imposes a strong inductive bias that matches the structure of visual data.

Exercise 4.6 (47.5 points)

One of the earliest and most notable CNNs is LeNet-5, created by a team led by Yann LeCun for the hand-written digit recognition task (MNIST). The LeNet-5 neural network has the following architecture, with the activation functions after each convolution and dense layer being hyperbolic tangent (`tanh`).



- How many parameters does the LeNet-5 network have? (7.5 points)
- Implement this CNN using JAX and Flax. You do not need to include the dropout technique in your network as LeNet-5 is not prone to overfitting due to its low parameter count. However, include the `adamw` optimizer in addition to batching. The Flax documentation provides how to implement convolution and pooling layers, in addition the expected input shape structure. (15 points)
- Train your implemented model for 10 epochs and report its accuracy results for 3 different seeds. Experiment with different learning rate and weight decays and report your findings. (15 points)
- How do these results compare to the MLP implementation of this task? How do you explain the accuracy differences (if there are any) despite their difference in parameter count? (10 points)